

TaxonVision-MLOps

Comprehensive Technical Implementation Guide

Systems Architecture, Infrastructure, and Codebase Reference

Benjamin Förster

October 11, 2026

Abstract

This technical report serves as an exhaustive, implementation-level architectural manual and code base reference for the TaxonVision-MLOps platform. Designed to document every structural component, configuration file, deployment manifest, mathematical loss formulation, and runtime daemon in the active repository, this document provides complete visibility into the physical server topology (`hive-mind` running Ubuntu 26.04.1 LTS, NVIDIA Driver 615.71.09, CUDA 13.4, and the NVIDIA GeForce RTX 5070 Ti with Blackwell SM 120), the lightweight Kubernetes (k3s) cluster partitioned into `jenkins-ci` and `taxon-vision` namespaces, the Cloudflare Zero Trust ingress tunnel, and the four-slot NVIDIA GPU time-slicing device plugin. Furthermore, this manual dissects the pure Pixi multi-environment toolchain, the automated DagsHub storage remote for Data Version Control (DVC) and MLflow tracking, the Cui et al. Class-Balanced Loss dynamics on biological long-tail Pareto distributions, the Optuna Bayesian hyperparameter optimisation engine, Split Conformal Prediction, Energy-Based Out-of-Distribution rejection, and the unified FastAPI and HTMX operator dashboard. Every configuration parameter, file extension, and container specification is strictly analysed based upon active codebase evidence.

Contents

Abstract	i
1 Repository Architecture, Directory Hierarchy, and File Extension Semantics	1
1.1 Root Directory Topology	1
1.2 Configuration Manifests and Semantics	1
1.3 Specialised File Formats and Storage Conventions	1
2 Bare-Metal Server Environment and Pixi Toolchain Interlock	2
2.1 Physical Compute Node Baseline	2
2.2 Multi-Target Toolchain and Pixi Environment Isolation	2
2.3 Command Orchestration via Justfile	2
3 Lightweight Kubernetes (k3s) Cluster Topology and Security RBAC	3
3.1 Cluster Architecture and Network Edge	3
3.2 Namespace Partitioning and Multi-Tenant Segregation	3
3.3 Role-Based Access Control (RBAC) Specification	3
4 GPU Virtualisation, NVIDIA Device Plugin, and VRAM Quotas	4
4.1 The Single-GPU Multi-Tenancy Challenge	4
4.2 NVIDIA Device Plugin Time-Slicing Configuration	4
4.3 Deterministic VRAM Budget Capping and OOM Prevention	4
5 Containerisation Architecture: Dockerfile and Composition Manifests	5
5.1 Multi-Stage Container Construction Strategy	5
5.2 Auxiliary Services and Docker Compose Topology	5
5.3 Container Hardening and Privilege Minimisation	5
6 Kubernetes Workloads, Service Manifests, and Rolling Updates	6
6.1 API Deployment Specification	6
6.2 Health Probes and Zero-Downtime Rolling Restarts	6
6.3 Kubernetes Service Abstraction and Routing	6
7 Declarative Jenkins Pipeline and Ephemeral Kubernetes Agents	7
7.1 Declarative Pipeline Architecture and Pod Template	7
7.2 Ephemeral Agent Lifecycle and Cache Volume Mounting	7
7.3 Stage-by-Stage Execution and Promotion Gates	7
8 Data Ingestion Engine, Zero-Copy Arrow Streaming, and License Audits	8
8.1 iNaturalist API Ingestion and Query Mechanics	8
8.2 Zero-Copy Apache Arrow Streaming Pipeline	8
8.3 Open License Verification and Attribution Retention	8
9 Data Version Control (DVC) and Remote Storage Topology	9
9.1 DVC Pipeline Graph and Cryptographic Manifests	9

9.2	DagsHub S3-Compatible Remote Storage Architecture	9
9.3	Synchronization Lifecycles in CI/CD and Local Workstations	9
10	Domain Modelling, DarwinCore Standards, and Taxonomic Trees	10
10.1	DarwinCore Taxon Schema and Catalog Implementation	10
10.2	Observation Entities and Spatiotemporal Metadata	10
10.3	License Verification Domain Invariants	10
11	Vision Backbone Factory and Representation Extraction Caching	11
11.1	Foundation Vision Backbone Portfolio	11
11.2	Extensible Feature Extractor Factory Architecture	11
11.3	Representation Extraction and Disk-Caching Strategy	11
12	Class-Balanced Loss Formulation and Long-Tail Gradient Dynamics	12
12.1	The Biological Long-Tail Pareto Distribution	12
12.2	Effective Number of Samples Mathematical Derivation	13
12.3	Weight Normalisation and Implementation Details	14
12.4	Integration with Classification Heads and Optuna Tuning	15
13	Classification Head Architecture, Optimization Loops, and Callbacks	16
13.1	Classification Head Geometry and Transfer Learning	16
13.2	Cached Embedding Training Loop and Mini-Batch Dynamics	16
13.3	Convergence Callbacks and Best-Weight Restoration	16
14	Bayesian Hyperparameter Optimization with Optuna TPE and ASHA	17
14.1	Bayesian Optimization Principles and TPE Mechanics	17
14.2	Search Space Parameterisation and Objective Formulation	17
14.3	Asynchronous Successive Halving (ASHA) Pruning in Practice	17
15	Experiment Tracking, Cryptographic Lineage, and MLflow Model Registry	18
15.1	Experiment Tracking Architecture on DagsHub	18
15.2	Deterministic Lifecycle Management via RAII Scopes	18
15.3	Cryptographic Lineage Capture and Automated Model Promotion	18
16	Epistemic Safety: Split Conformal Prediction and Energy-Based OOD Rejection	19
16.1	The Epistemic Safety Problem in Biological Vision	19
16.2	Split Conformal Prediction and Statistical Coverage Bounds	19
16.3	Helmholtz Free Energy Out-of-Distribution Detection	19
17	Active Learning Strategy and Priority Triage Queue	20
17.1	Uncertainty-Based Query Acquisition Mechanics	20
17.2	Priority Triage Queue Implementation	20
17.3	Closing the Continuous Retraining Annotation Loop	20
18	ONNX Export, Zero-Allocation Runtime, and INT8 Quantization	21
18.1	Computational Graph Export and Dynamic Axes	21
18.2	Zero-Allocation C++ Inference Execution Engine	21
18.3	INT8 Quantization Calibration and Class Activation Mapping	21
19	FastAPI REST Service and Server-Rendered HTMX Operator Dashboard	22
19.1	Unified REST and Web Application Architecture	22
19.2	API Schemas, Endpoints, and Inference Routing	22
19.3	Server-Rendered HTMX Operator Interface	22
20	Real-Time Telemetry, Prometheus Instrumentation, and Drift Detection	23
20.1	Prometheus Telemetry Instrumentation and Metrics Design	23
20.2	Serving Latency and Conformal Set Monitoring	23

20.3	Embedding Representation Drift via Wasserstein and MMD	23
21	Quality Assurance: Pytest Matrix, Invariant Testing, and Complexity Budgets	24
21.1	Testing Architecture and Test Matrix Topology	24
21.2	Mathematical Invariant Assertions and Property-Based Testing	24
21.3	Cognitive Complexity Budgets and Pre-Commit Quality Gates	24
22	Open Knowledge Format (OKF v0.2) and Master Agent Routing	25
22.1	Open Knowledge Format (OKF v0.2) Schema and Trust Tiers	25
22.2	Multi-Agent Governance and Decision Routing Matrices	25
22.3	Automated Validation Scripts and D3 Knowledge Graph Visualization	25
23	Developer Operations, Environment Setup, and Engineering Runbooks	26
23.1	Developer Workstation Onboarding and Environment Setup	26
23.2	Dataset Synchronization and Remote Storage Runbook	26
23.3	Cluster Operations, GPU Debugging, and Emergency Rollback	26

Chapter 1

Repository Architecture, Directory Hierarchy, and File Extension Semantics

1.1 Root Directory Topology

1.2 Configuration Manifests and Semantics

1.3 Specialised File Formats and Storage Conventions

Chapter 2

Bare-Metal Server Environment and Pixi Toolchain Interlock

2.1 Physical Compute Node Baseline

2.2 Multi-Target Toolchain and Pixi Environment Isolation

2.3 Command Orchestration via Justfile

Chapter 3

Lightweight Kubernetes (k3s) Cluster Topology and Security RBAC

- 3.1 Cluster Architecture and Network Edge
- 3.2 Namespace Partitioning and Multi-Tenant Segregation
- 3.3 Role-Based Access Control (RBAC) Specification

Chapter 4

GPU Virtualisation, NVIDIA Device Plugin, and VRAM Quotas

- 4.1 The Single-GPU Multi-Tenancy Challenge
- 4.2 NVIDIA Device Plugin Time-Slicing Configuration
- 4.3 Deterministic VRAM Budget Capping and OOM Prevention

Chapter 5

Containerisation Architecture: Dockerfile and Composition Manifests

- 5.1 Multi-Stage Container Construction Strategy
- 5.2 Auxiliary Services and Docker Compose Topology
- 5.3 Container Hardening and Privilege Minimisation

Chapter 6

Kubernetes Workloads, Service Manifests, and Rolling Updates

6.1 API Deployment Specification

6.2 Health Probes and Zero-Downtime Rolling Restarts

6.3 Kubernetes Service Abstraction and Routing

Chapter 7

Declarative Jenkins Pipeline and Ephemeral Kubernetes Agents

- 7.1 Declarative Pipeline Architecture and Pod Template
- 7.2 Ephemeral Agent Lifecycle and Cache Volume Mounting
- 7.3 Stage-by-Stage Execution and Promotion Gates

Chapter 8

Data Ingestion Engine, Zero-Copy Arrow Streaming, and License Audits

- 8.1 iNaturalist API Ingestion and Query Mechanics
- 8.2 Zero-Copy Apache Arrow Streaming Pipeline
- 8.3 Open License Verification and Attribution Retention

Chapter 9

Data Version Control (DVC) and Remote Storage Topology

- 9.1 DVC Pipeline Graph and Cryptographic Manifests
- 9.2 DagsHub S3-Compatible Remote Storage Architecture
- 9.3 Synchronization Lifecycles in CI/CD and Local Workstations

Chapter 10

Domain Modelling, DarwinCore Standards, and Taxonomic Trees

- 10.1 DarwinCore Taxon Schema and Catalog Implementation
- 10.2 Observation Entities and Spatiotemporal Metadata
- 10.3 License Verification Domain Invariants

Chapter 11

Vision Backbone Factory and Representation Extraction Caching

11.1 Foundation Vision Backbone Portfolio

11.2 Extensible Feature Extractor Factory Architecture

11.3 Representation Extraction and Disk-Caching Strategy

Chapter 12

Class-Balanced Loss Formulation and Long-Tail Gradient Dynamics

12.1 The Biological Long-Tail Pareto Distribution

In the domain of biological observation, data categorisation, and large-scale ecological monitoring, empirical datasets exhibit an inherent and inescapable power-law, or Zipfian, distribution. This statistical phenomenon represents a fundamental structural and mathematical challenge in training robust, large-scale computer vision models. The observation frequencies of taxa span several orders of magnitude, decaying inversely proportional to their rank. At the uppermost echelon of this distribution lie the head classes, comprising common, geographically ubiquitous, or often urban-dwelling species such as universally recognised birds, plants, and synanthropic insects. These abundant taxa routinely accumulate tens or hundreds of thousands of distinct, high-quality visual observations. At the opposite extreme stretches the profound biological long-tail, encapsulating rare, elusive, geographically restricted, or critically endangered taxa. These minority classes frequently register fewer than five verifiable observations across the entire global monitoring network.

When optimising deep neural networks on such extreme Pareto distributions using the standard unweighted Cross-Entropy loss criterion, the gradient descent dynamics become severely pathological. In a mini-batch stochastic gradient descent (SGD) environment, the expected value of the loss gradient is overwhelmingly dominated by the vast numerical superiority of the majority classes. During the optimisation trajectory, the feature representations of the minority classes suffer from catastrophic collapse. The network, driven by the geometry of the loss landscape, implicitly learns to ignore the rare taxa. It effectively collapses the decision boundaries of minority classes into the continuous feature space already occupied by the dominant species, thereby maximising overall top-1 accuracy at the direct, unrecoverable expense of minority class recall. This representation learning collapse renders the resulting model ecologically invalid for biodiversity assessment, as the rare taxa are almost universally those of the greatest conservation and scientific interest.

A classical countermeasure to this structural imbalance involves the application of naive inverse class frequency weighting. In this theoretical paradigm, the loss contribution of each discrete observation is linearly scaled by the reciprocal of the total observation count for its corresponding ground-truth class. However, this approach introduces an opposite, yet equally destructive, mathematical pathology. The reciprocal scaling mechanism generates extreme gradient magnitude variance for the rarest outliers. A single noisy, atypical, or mislabelled observation belonging to a rare taxon, when amplified by a massive inverse frequency multiplier, produces explosive and highly unstable gradient updates. These volatile updates systematically destabilise the momentum buffers of modern adaptive optimisers, such as Adam and AdamW. The resultant non-convergent, oscillatory training dynamics fail to discover generalised minima. Consequently, neither the unweighted baseline nor the naive inverse frequency paradigm offers a mathematically sound framework for

optimising deep networks over ecological long-tail distributions.

12.2 Effective Number of Samples Mathematical Derivation

To resolve the irreconcilable dichotomy between gradient domination by head classes and gradient explosion originating from rare classes, Cui et al. (CVPR 2019) introduced a mathematically rigorous framework predicated on the volume of data overlap in continuous feature space. The core thesis of this framework posits that as the number of samples in a specific class increases, the marginal informational value of each additional sample diminishes monotonically. This phenomenon is analytically modelled via a random walk volume overlap process operating within the high-dimensional feature manifold of the deep neural network.

Instead of treating each discrete observation as an independent, strictly non-overlapping unit of information, the formulation defines a continuous effective number of samples, denoted mathematically as E_N . The theoretical assumption is that the total volume of the feature space encompassing all possible semantic and visual variations of a specific biological taxon is finite. A single sample covers a small sub-volume within this bounded space. Subsequent samples drawn independently from the identical class distribution will, with increasing geometric probability, fall into a region of the feature space that is already covered by the union of previous samples. The expected volume of this union of subsets provides the mathematical basis for the effective sample size, bounding the influence of hyper-abundant classes.

Given an empirical count of N_i training examples for a specific category or class i , the effective number of samples is formally derived as a geometric series:

$$E_{N_i} = \frac{1 - \beta^{N_i}}{1 - \beta}$$

where the hyperparameter $\beta \in [0, 1)$ quantifies the feature space volume overlap. Conceptually, β dictates the precise geometric rate at which the marginal information of continuously added samples decays. In typical large-scale vision applications, β is closely bounded near unity, conventionally taking values within the logarithmic range $[0.9, 0.9999]$.

The profound theoretical elegance of this formulation lies in its seamless, continuous interpolation between uniform weighting and inverse frequency weighting. We can rigorously examine the asymptotic behaviour at the two extremal boundaries of the hyperparameter β .

First, consider the lower limit as $\beta \rightarrow 0$. In this scenario, the volume overlap is defined as absolute zero, implying that every collected sample provides completely novel, mutually exclusive, and non-overlapping information. The effective sample size expression elegantly evaluates to unity:

$$\lim_{\beta \rightarrow 0} E_{N_i} = 1$$

Under this boundary condition, the calculated effective volume is perfectly uniform across all taxonomic classes, operating completely regardless of their empirical observation counts. The subsequent weighting mechanism thus collapses precisely back to the standard uniform, unweighted Cross-Entropy formulation.

Second, consider the upper limit as $\beta \rightarrow 1$. In this case, the overlap approaches absolute unity, and we must apply L'Hôpital's rule to formally evaluate the indeterminate limit:

$$\lim_{\beta \rightarrow 1} E_{N_i} = \lim_{\beta \rightarrow 1} \frac{\frac{\partial}{\partial \beta}(1 - \beta^{N_i})}{\frac{\partial}{\partial \beta}(1 - \beta)} = \lim_{\beta \rightarrow 1} \frac{-N_i \beta^{N_i-1}}{-1} = N_i$$

As β converges asymptotically towards 1, the effective number of samples converges directly to the raw, unmodified empirical count N_i . In this specific operational regime, weighting the loss criterion

by the reciprocal of the effective sample size perfectly recovers the classical inverse class frequency weighting. Therefore, the parameter β provides a continuous, highly controllable spectrum of mathematical regularisation. It empowers the optimisation landscape to balance the informational geometry of the long-tail distribution safely between the catastrophic extremes of feature collapse and gradient explosion.

12.3 Weight Normalisation and Implementation Details

Translating this continuous theoretical model into a robust PyTorch tensor operation requires exceptionally strict attention to numerical stability, gradient scaling, and deterministic execution semantics. The optimal class weight W_i for a given taxonomic class i is defined as strictly inversely proportional to its mathematical effective sample volume:

$$W_i = \frac{1 - \beta}{1 - \beta^{N_i} + \epsilon}$$

where $\epsilon = 10^{-8}$ is a strictly small positive constant injected purposefully into the denominator to proactively prevent division-by-zero singularities. This regularisation is particularly critical in streaming, data-parallel, or dynamic dataset contexts. In active learning architectures, certain highly rare classes might temporarily register zero observations during specific training folds or cross-validation splits, making this numerical safeguard an indispensable operational requirement.

Furthermore, a critical mathematical normalisation constraint must be rigorously enforced across the entire set of all classes to fundamentally preserve the total gradient magnitude scale of the loss function. If the dynamically computed weights are applied naively without geometric normalisation, the overall magnitude of the computed loss and consequently the magnitude of the backward gradients propagated through the layers of the network will be arbitrarily scaled by the empirically chosen value of β . To systematically counteract this scalar distortion, the raw weights are strictly normalised such that their linear sum precisely equals the total number of distinct classes K :

$$\sum_{i=1}^K W_i = K$$

This rigid normalisation protocol ensures that the arithmetic mean class weight remains strictly equal to 1.0. By anchoring the mean weight to unity, the global learning rate calibration of the chosen optimiser is preserved unconditionally across different values of β . In the TaxonVision codebase, specifically within the module `src/taxon_vision/models/loss.py`, the `ClassBalancedLoss` class architecture enforces this constraint algorithmically during the instantiation lifecycle.

To guarantee numerical stability during the initialisation phase, the empirical sample counts are ingested as an array of double-precision floating-point numbers (`numpy.float64`). To further safeguard against undefined mathematical behaviour, the minimum operational sample count is clamped strictly to 1.0 via the `numpy.maximum` primitive before any exponential operations are executed. The final, mathematically normalised weights are subsequently cast down to single-precision `torch.float32` tensors for hardware execution.

Crucially, these static weights are registered as a persistent tensor buffer within the `torch.nn.Module` hierarchy using the standard `register_buffer` directive. This critical architectural decision ensures that the pre-calculated static class weights are seamlessly and automatically migrated to the active physical execution device, whether that be a CUDA GPU cluster or an Apple Silicon MPS endpoint, directly alongside the trainable model parameters during distributed training orchestration. Simultaneously, this registration paradigm prevents the weights from being erroneously treated as trainable parameters and updated by the optimiser’s backward pass. The actual objective loss is then computed with peak hardware efficiency using the native `torch.nn.functional.cross_entropy` function, which seamlessly accepts the pre-computed `device_weights` tensor via its keyword arguments.

12.4 Integration with Classification Heads and Optuna Tuning

The robust, large-scale utilisation of the Class-Balanced Loss mandates flawless dynamic parameter integration within the automated operational training pipelines. Within the `taxon_vision.models.trainer` sub-package, the initialisation of the loss module requires the exact, deterministic extraction of categorical population statistics. During the construction of the computational training graph, the sequence `samples_per_class` is dynamically computed and extracted from the immutable parquet file metadata of the current training data split, canonically represented by the `EmbeddingSplit` object. This deterministic, programmatic linkage guarantees absolutely that the loss weights accurately and exclusively reflect the empirical distribution of the exact data subset ingested by the current epoch. It completely circumvents any potential statistical data leakage from held-out validation or test splits, ensuring epistemically sound evaluation metrics.

The rigorous determination of the mathematically optimal volume overlap parameter β is highly non-trivial. The exact optimal scalar value is extraordinarily sensitive to the specific taxonomic resolution, geographical scope, and dataset geometry of the operational deployment. Consequently, β is treated as a primary, top-level tunable hyperparameter within the broader architectural optimisation strategy. Within the specific script `src/taxon_vision/models/training/tuning.py`, automated Bayesian hyperparameter optimisation is orchestrated via the highly robust Optuna framework. The multidimensional search space for β is constructed as a continuous log-uniform distribution, typically and strictly bounded between the values 0.99 and 0.9999. This specific logarithmic range deliberately focuses the search algorithm’s sampling density on the extreme asymptotic regime where the most critical variance-bias trade-offs manifest for biological taxonomic data.

The Optuna `HyperparameterObjective` function dynamically couples the Tree-structured Parzen Estimator (TPE) probabilistic sampling algorithm with the aggressive Asynchronous Successive Halving Algorithm (ASHA) early stopping pruner. During the continuous automated search process, candidate configurations of β , evaluated in precise conjunction with the network learning rate, the weight decay magnitude, and the regularisation dropout rate, are iteratively evaluated in isolated execution trials. The classification linear head, parametrised explicitly with the dynamically instantiated `ClassBalancedLoss`, is trained natively on pre-computed, cached ONNX representation embeddings.

To strictly enforce computational efficiency budgets, the integrated ASHA pruner rapidly and deterministically terminates underperforming hyperparameter trials based on their intermediate validation Macro PR-AUC scores. This metric, the Area Under the Precision-Recall Curve calculated via strict macro-averaging, is specifically and exclusively chosen as the terminal objective function. Unlike standard overall micro-accuracy, Macro PR-AUC remains strictly unbiased in the presence of extreme class imbalance. It heavily and proportionately penalises models that collapse their predictive distributions on minority taxa, ensuring that algorithmic generalisation extends safely deep into the long-tail.

The entire hyperparameter trajectory across the Optuna search, including the specific configuration of the effective sample size loss weighting parameters, is serially tracked by the central MLflow telemetry server. This comprehensive serialisation provides full historical lineage, ensuring total auditability and reproducibility for the optimisation sequence of the taxonomic classifier. Through this intricate, deterministic integration of the effective sample size mathematics with aggressive Bayesian tuning algorithms, the overall pipeline guarantees mathematically provable, Pareto-optimal performance across both the ubiquitous head classes and the critically endangered biological species located in the deepest extreme of the power-law tail.

Chapter 13

Classification Head Architecture, Optimization Loops, and Callbacks

13.1 Classification Head Geometry and Transfer Learning

13.2 Cached Embedding Training Loop and Mini-Batch
Dynamics

13.3 Convergence Callbacks and Best-Weight Restoration

Chapter 14

Bayesian Hyperparameter Optimization with Optuna TPE and ASHA

- 14.1 Bayesian Optimization Principles and TPE Mechanics
- 14.2 Search Space Parameterisation and Objective Formulation
- 14.3 Asynchronous Successive Halving (ASHA) Pruning in Practice

Chapter 15

Experiment Tracking, Cryptographic Lineage, and MLflow Model Registry

- 15.1 Experiment Tracking Architecture on DagsHub
- 15.2 Deterministic Lifecycle Management via RAII Scopes
- 15.3 Cryptographic Lineage Capture and Automated Model Promotion

Chapter 16

Epistemic Safety: Split Conformal Prediction and Energy-Based OOD Rejection

- 16.1 The Epistemic Safety Problem in Biological Vision
- 16.2 Split Conformal Prediction and Statistical Coverage Bounds
- 16.3 Helmholtz Free Energy Out-of-Distribution Detection

Chapter 17

Active Learning Strategy and Priority Triage Queue

- 17.1 Uncertainty-Based Query Acquisition Mechanics
- 17.2 Priority Triage Queue Implementation
- 17.3 Closing the Continuous Retraining Annotation Loop

Chapter 18

ONNX Export, Zero-Allocation Runtime, and INT8 Quantization

18.1 Computational Graph Export and Dynamic Axes

18.2 Zero-Allocation C++ Inference Execution Engine

18.3 INT8 Quantization Calibration and Class Activation Mapping

Chapter 19

FastAPI REST Service and Server-Rendered HTMX Operator Dashboard

- 19.1 Unified REST and Web Application Architecture
- 19.2 API Schemas, Endpoints, and Inference Routing
- 19.3 Server-Rendered HTMX Operator Interface

Chapter 20

Real-Time Telemetry, Prometheus Instrumentation, and Drift Detection

- 20.1 Prometheus Telemetry Instrumentation and Metrics Design
- 20.2 Serving Latency and Conformal Set Monitoring
- 20.3 Embedding Representation Drift via Wasserstein and MMD

Chapter 21

Quality Assurance: Pytest Matrix, Invariant Testing, and Complexity Budgets

- 21.1 Testing Architecture and Test Matrix Topology
- 21.2 Mathematical Invariant Assertions and Property-Based Testing
- 21.3 Cognitive Complexity Budgets and Pre-Commit Quality Gates

Chapter 22

Open Knowledge Format (OKF v0.2) and Master Agent Routing

- 22.1 Open Knowledge Format (OKF v0.2) Schema and Trust Tiers
- 22.2 Multi-Agent Governance and Decision Routing Matrices
- 22.3 Automated Validation Scripts and D3 Knowledge Graph Visualization

Chapter 23

Developer Operations, Environment Setup, and Engineering Runbooks

- 23.1 Developer Workstation Onboarding and Environment Setup
- 23.2 Dataset Synchronization and Remote Storage Runbook
- 23.3 Cluster Operations, GPU Debugging, and Emergency Rollback

Bibliography

- [1] Anastasios N. Angelopoulos and Stephen Bates. A gentle introduction to conformal prediction and distribution-free uncertainty quantification. *arXiv preprint arXiv:2107.07511*, 2021.
- [2] Jordan T. Ash, Chicheng Zhang, Akshay Krishnamurthy, John Langford, and Alekh Agarwal. Deep batch active learning by diverse gradient embeddings. In *International Conference on Learning Representations (ICLR)*, 2020.
- [3] Yin Cui, Menglin Jia, Tsung-Yi Lin, Yang Song, and Serge Belongie. Class-balanced loss based on effective number of samples. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 9268–9277. IEEE, 2019.
- [4] Pengzhen Ren, Yun Xiao, Xiaojun Chang, Po-Yao Huang, Zhihui Li, Brij B. Gupta, Xiaojiang Chen, and Xinwang Wang. A survey of deep active learning. *ACM Computing Surveys (CSUR)*, 54(9):1–40, 2021.
- [5] Samuel Stevens, Jiaman Wu, Matthew J. Thompson, Elizabeth G. Campolongo, Chan Hee Song, David E. Carlyn, Li Dong, Wasila M. Dahdul, Charles Stewart, Tanya Berger-Wolf, Wei-Lun Chao, and Yu Su. Bioclip: A vision foundation model for the tree of life. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 19412–19424. IEEE, 2024.
- [6] Grant Van Horn, Oisin Mac Aodha, Yang Song, Yin Cui, Chen Sun, Alex Shepard, Hartwig Adam, Pietro Perona, and Serge Belongie. The inaturalist species classification and detection dataset. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8769–8778. IEEE, 2018.